



Using NamedParameterJdbcTemplate

boot-jdbc-demo7: Files required

1. MyBootApplication.java	Updated in Demo7
2. CustomerDAO.java	Updated in Demo7
3. CustomerDAOImpl.java	Updated in Demo7
4. Customer.java	Same As Demo6
5. CustomerRowMapper.java	Same As Demo6
6. application.properties	Updated in Demo7
7. pom.xml	Updated in Demo7

1) MyBootApplication.java

```
package com.jlcindia.spring;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

@SpringBootApplication
public class MyBootApplication implements CommandLineRunner {

    @Autowired
    CustomerDAO custDAO;

    @Override
    public void run(String... args) throws Exception {

        // 1.Add the Customer
        Customer cust = new Customer(107, "aaa", "aaa@jlc", 111, "Delhi");
        custDAO.addCustomer(cust);

        // 2.get All Customers
        List<Customer> mylist1 = custDAO.getAllCustomers();
        mylist1.forEach(mycust -> System.out.println(mycust));
        System.out.println("-----");
    }
}
```



// 3.Get Customers By City

```
List<Customer> mylist2 = custDAO.getCustomersByCity("Delhi");  
mylist2.forEach(mycust -> System.out.println(mycust));  
System.out.println("-----");
```

// 4.Get Customers Email By Phone

```
String email = custDAO.getCustomerEmailByPhone(111);  
System.out.println(email);  
System.out.println("Done!!!");  
}
```

```
public static void main(String[] args) {  
    SpringApplication.run(MyBootApplication.class, args);  
}  
}
```

2)CustomerDAO.java

```
package com.jlcindia.spring;  
  
import java.util.List;  
import java.util.Map;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
public interface CustomerDAO {  
    public void addCustomer(Customer cust);  
    public void addCustomer(Map<String,Object> map);  
    public List<Customer> getAllCustomers();  
    public List<Customer> getCustomersByCity(String city);  
    public String getCustomerEmailByPhone(int phone);  
}
```

3) CustomerDAOImpl.java

```
package com.jlcindia.spring;  
  
import java.util.*;  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.jdbc.core.namedparam.NamedParameterJdbcTemplate;  
import org.springframework.stereotype.Repository;  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */
```



@Repository

```
public class CustomerDAOImpl implements CustomerDAO{
```

@Autowired

```
NamedParameterJdbcTemplate namedJTemp;
```

@Autowired

```
JdbcTemplate jtemp;
```

@Override

```
public void addCustomer(Customer cust) {
```

```
String SQL="insert into mycustomers values(:mycid,:mycname,:myemail,:myphone,:mycity)";
```

```
Map<String,Object> params = new HashMap<>();
```

```
params.put("mycid", cust.getCid());
```

```
params.put("mycname", cust.getCname());
```

```
params.put("myemail", cust.getEmail());
```

```
params.put("myphone", cust.getPhone());
```

```
params.put("mycity", cust.getCity());
```

```
namedJTemp.update(SQL, params);
```

```
}
```

@Override

```
public void addCustomer(Map<String, Object> map) {
```

```
String SQL="insert into mycustomers values(:mycid,:mycname,:myemail,:myphone,:mycity)";
```

```
namedJTemp.update(SQL, map);
```

```
}
```

@Override

```
public List<Customer> getAllCustomers() {
```

```
String SQL="select * from mycustomers";
```

```
List<Customer> mylist = namedJTemp.query(SQL, new CustomerRowMapper());
```

```
return mylist;
```

```
}
```

@Override

```
public List<Customer> getCustomersByCity(String city) {
```

```
String SQL="select * from mycustomers where city=:mycity";
```

```
Map<String,Object> params = new HashMap<>();
```

```
params.put("mycity", city);
```

```
List<Customer> mylist = namedJTemp.query(SQL, params,new CustomerRowMapper());
```

```
return mylist;
```

```
}
```



```
public String getCustomerEmailByPhone(int phone) {  
String SQL="select email from mycustomers where phone=:myphone";  
  
Map<String,Object> params = new HashMap<>();  
params.put("myphone", phone);  
  
String email = namedJTemp.queryForObject(SQL, params, String.class);  
return email;  
}  
  
}
```

6)application.properties

#Database Properties

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/myjlcdb5  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

#Hikari Properties

```
spring.datasource.hikari.maximum-pool-size=100  
spring.datasource.hikari.idle-timeout=20000
```

#Log Properties

```
logging.level.root=DEBUG  
logging.pattern.console=%-5level %logger{36} - %m %n
```

7)pom.xml

```
<project ...>
```

```
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.springboot</groupId>  
<artifactId>boot-jdbc-demo7</artifactId>  
<version>1.0</version>
```

```
<name>boot-jdbc-demo7</name>  
<description>This is My Spring Boot Application</description>  
<url>https://www.jlcindia.com</url>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```

Q24) What types of Parameters can be used when we write the Queries?

Q25) When Can I Use JdbcTemplate?

Q26) When Can I Use NamedParameterJdbcTemplate?